



Understanding Templates in Ansible

Generating Dynamic Configuration Files

Introduction

In many IT automation tasks, we need to manage configuration files dynamically. Instead of manually editing files for different servers, Ansible allows us to use **Templates** to create **customized configuration files** for each system.

Ansible Templates use **Jinja2**, a powerful templating engine, to generate files dynamically based on variables and conditions.

This guide will cover:

- What are Templates in Ansible?
 - Why use Templates?
 - How to create and use Templates in Playbooks
 - Jinja2 syntax and filters
 - Common troubleshooting issues
-

What are Templates in Ansible?

Templates in Ansible are **Jinja2-based files (.j2)** that help generate **custom configuration files** dynamically. Instead of using **static text**, Templates allow:

- Using variables inside configuration files
- Applying conditional logic
- Looping through data
- Reducing manual changes across multiple servers



*Think of a Template as a **blueprint** that dynamically generates configuration files for different servers!*



Why Use Templates in Ansible?

Feature	Benefit
Dynamic Configuration	Generate custom settings per server.
Variables Support	Replace static values with variables.
Conditional Logic	Create different outputs based on conditions.
Looping Support	Easily manage multiple entries.
Reduces Manual Work	Eliminates repetitive file modifications.

💡 *Templates make automation **more flexible and scalable!***

Creating and Using Templates in Playbooks

1. Create a Template File

Templates are stored inside the **templates/ directory** and have a **.j2** extension.

Example: Create **templates/nginx.conf.j2**:

```
server {  
    listen 80;  
    server_name {{ server_name }};  
    root /var/www/{{ site_folder }};  
  
    location / {  
        index index.html;
```



```
}  
  
}
```

2. Use the Template in a Playbook

Define variables in the Playbook:

```
---  
  
- name: Deploy Web Server Configuration  
  hosts: webservers  
  become: yes  
  vars:  
    server_name: example.com  
    site_folder: mywebsite  
  
  tasks:  
    - name: Copy Nginx Configuration File  
      template:  
        src: templates/nginx.conf.j2  
        dest: /etc/nginx/sites-available/default  
      notify: Restart Nginx  
  
  handlers:  
    - name: Restart Nginx  
      service:  
        name: nginx  
        state: restarted
```



3. Run the Playbook

```
ansible-playbook -i inventory.ini deploy-nginx.yml
```

💡 This will generate a **customized Nginx configuration file** based on the variables **server_name** and **site_folder**.

Using Jinja2 in Ansible Templates

Jinja2 is a powerful templating engine used by Ansible to handle **variables**, **conditions**, **loops**, and **filters**.

1. Using Variables in Templates

You can insert variables inside a Template using **{{ variable_name }}**.

```
server_name {{ server_name }};
```

2. Conditional Logic in Templates

Templates support **if** conditions to modify content dynamically.

```
server {  
    listen 80;  
    {% if ssl_enabled %}  
    listen 443 ssl;  
    ssl_certificate /etc/nginx/ssl/{{ server_name }}.crt;  
    ssl_certificate_key /etc/nginx/ssl/{{ server_name }}.key;  
    {% endif %}  
  
    server_name {{ server_name }};  
}
```



If `ssl_enabled` is `true`, the SSL configuration will be added; otherwise, it won't appear in the file.

3. Loops in Templates

Loops help generate **multiple entries dynamically**.

Example: Generate a list of allowed IPs:

```
allow 127.0.0.1;
{% for ip in allowed_ips %}
allow {{ ip }};
{% endfor %}
```

Define `allowed_ips` in Playbook:

```
vars:
  allowed_ips:
    - 192.168.1.100
    - 192.168.1.101
```

💡 *Loops reduce repetitive entries and improve maintainability!*

Filtering Data in Templates

Jinja2 **filters** modify variables before displaying them.

Filter	Purpose	Example
<code>lower</code>	Converts text to lowercase	<code>{{ server_name lower }}</code>



Filter	Purpose	Example
upper	Converts text to uppercase	<code>{{ server_name upper }}</code>
replace	Replaces text	<code>{{ server_name replace("example", "test") }}</code>
default	Sets a default value	<code>{{ server_name default("localhost") }}</code>

💡 *Filters help clean and modify data dynamically!*

Managing Secrets in Templates

Templates can fetch **sensitive data** securely using **Ansible Vault**.

1. Encrypt Sensitive Data

```
ansible-vault encrypt secrets.yml
```

2. Use Vault Data in Templates

Example: Fetching a password inside a Template.

```
db_password: "{{ vault_db_password }}"
```

💡 *Using **Vault** protects confidential information like passwords and API keys!*

Common Issues and Troubleshooting

Problem	Solution
Undefined variable	Ensure the variable is defined in the Playbook .



Problem	Solution
Indentation error	Verify YAML and Jinja2 indentation.
Missing file or directory	Ensure the Template file exists in the correct location.
File not updating	Run Playbook with --diff to check changes.

Know More (FAQs)

1. How do I preview the output of a Template before applying it?

Run:

```
ansible localhost -m template -a "src=templates/nginx.conf.j2  
dest=/tmp/nginx.conf" --check
```

2. Can I use multiple variables inside a Template?

Yes! You can use as many variables as needed.

```
server {  
    listen 80;  
    server_name {{ domain_name }};  
    root /var/www/{{ project_name }};  
}
```

3. How do I debug a Template?

Use the **debug** module in a Playbook:



tasks:

- name: Debug variable output

debug:

msg: "Server Name: {{ server_name }}"

4. Can Templates be used for non-configuration files?

Yes! Templates can generate **scripts**, **HTML files**, **JSON files**, and **more**.

Example: Generating a **Bash script** dynamically.

```
#!/bin/bash  
  
echo "Server Name: {{ server_name }}"
```

5. How do I enforce strict error checking in Templates?

Use **set -e** in scripts and **Jinja2 strict mode**.

```
{% if server_name is not defined %}  
ERROR: Missing server_name variable!  
{% endif %}
```

Conclusion

Ansible **Templates** allow the **dynamic generation of configuration files**, making automation more **flexible, scalable, and efficient**. They eliminate manual file editing and ensure **consistent configurations across multiple servers**.

👉 Try creating a **customized configuration file** using Ansible Templates today!